es6 unique array of objects with set

Asked 6 years, 11 months ago Modified 2 months ago Viewed 37k times

25

I came across this example for creating unique arrays with es6

```
[ ...new Set(array) ]
```

Which seems to work fine until I tried it with an array of objects and it didn't return unique array.

i.e.

```
let item = [ ...new Set([{id:123,value:'test'},{id:123,value:'test'}]) ];
```

Why is that ?

javascript arrays ecmascript-6

Share Improve this question Follow edited Oct 12, 2016 at 11:10

asked Oct 12, 2016 at 11:07
StevieB
6,283 • 38 • 108 • 193

What do you mean by "doesnt work"? `[ ... new Set([{ val: 1 }, { val: 2 }, { val: 3 }]) ]` gives me an array of objects... – James Monger Oct 12, 2016 at 11:09

by Using Set I was hoping the get a filtered Unique array back – StevieB Oct 12, 2016 at 11:10

6 This has nothing to do with `Set`, two objects are **never** the same, even if they contain the same values – adeneo Oct 12, 2016 at 11:15

4 Answers

Sorted by: Highest score (default)

26

you can try to do

```
uniqueArray = a => [...new Set(a.map(o => JSON.stringify(o)))].map(s => JSON.parse(s))
```

I know its ugly as hell but in most cases works apart from where you have new Date() in your object param then that on stringify be converted to ISO string.

so then do

```
let arr = [{id:1},{id:1},{id:2}];
uniqueArray(arr) //[{id:1},{id:2}]
```

Share Improve this answer Follow edited Mar 1, 2018 at 13:31

answered Mar 1, 2018 at 13:24

Vic
414 ● 4 ● 7

I came across this question and I realised I've written exaclty the same with exactly the same syntax in my console :-) – ekqnp Dec 4, 2018 at 13:33

However, this solution won't work with regexp: `a = [{b: /foo/},{b: /bar/}]` will return `[{b:{}}]` – ekqnp Dec 4, 2018 at 15:33

Only this one is woking with Mongoose document. – Homan Huang Sep 1, 2019 at 11:21

20

> Why is that ?

As per documentation

> The Set object lets you store unique values of any type, whether primitive values or **object references**.

Now reference for each of those arrays inside that `Set` constructor will be different so they are not considered to be a unique value by the constructor.

Share Improve this answer Follow

answered Oct 12, 2016 at 11:13

gurvinder372
67k ● 10 ● 72 ● 94

5

This will work:

```
let objectReference = {id:123,value:'test'}
let uniqueArray = [...new Set([objectReference, objectReference])]

>> [{id:123,value:'test'}]
```

What you're doing:

```
let objRef1 = {id:123,value:'test'} // creates a reference to a location in memory
let objRef2 = {id:123,value:'test'} // creates a new reference to a different place in
memory

let uniqueArray = [...new Set([objRef1, objRef2])]
```

```
>> [{id:123,value:'test'},{id:123,value:'test'}]
```

Share Improve this answer Follow

answered Apr 27, 2017 at 20:13

Travis Heeter
13.1k ● 13 ● 87 ● 131

0

If you don't use a library like lodash or radash for example. You can use Set as answered by @Vic. Just a bug detected is about objects without same keys order. For example **{a: '1', b: '2'}** and **{b: '2', a: '1'}** are not string equals. Here is a working solution with covered cases

Implementation

```
uniq(source) {
  if (!Array.isArray(source)) {
     return [];
  }
  return [...new Set(source.map(o => {
    const sortedObjectKeys = Object.keys(o).sort();
    const obj = Object.assign({}, ...sortedObjectKeys.map(k => ({[k]: o[k]})) as any);
    return JSON.stringify(obj);
  }))]
  .map(s => JSON.parse(s));
}
```

Test cases

```
describe(`uniq`, () => {
  it('should return unique collection values', () => {
    expect(uniq([{v: 1}, {v: 2}, {v: 1}])).toEqual([{v: 1}, {v: 2}]);
  });

 it('should return unique collection values for unsorted properties', () => {
   expect(uniq([{a: 'test', v: 1}, {v: 2}, {v: 1, a: 'test'}])).toEqual([{a: 'test', v:
1}, {v: 2}]);
 });

 it('should return empty array for non array source', () => {
   expect(uniq({v: 1})).toEqual([]);
   expect(uniq('aString')).toEqual([]);
   expect(uniq(125)).toEqual([]);
   expect(uniq(true)).toEqual([]);
   expect(uniq([])).toEqual([]);
   expect(uniq(undefined)).toEqual([]);
   expect(uniq(null)).toEqual([]);
});

});
```